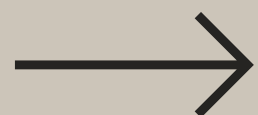


5 hooks que nadie te enseña y te diferencian del 99%



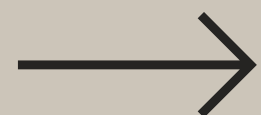
Adrián García Chavero

Construyo productos digitales
que dejan huella 🧠



useFormStatus

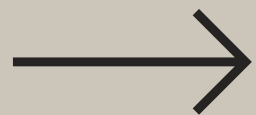
- Te permite **conocer el estado** de envío del formulario
- **Se acabaron los useState** para los *loading* manuales
- Muy útil para **Server Actions** (ellos mismos lo aconsejan)



Ejemplo de código

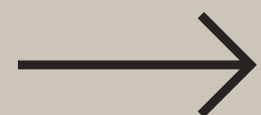
```
1 import { useFormStatus } from 'react-dom'
2 import { Button } from '@components/ui/button'
3
4 function SubmitButton() {
5   const { pending } = useFormStatus(); // el solito detecta si el formulario está enviando
6
7   return (
8     <Button type="submit" disabled={pending}>
9       {pending ? 'Enviando...' : 'Enviar'}
10    </Button>
11  )
12 }
13
14 export default function Form() {
15   async function submit(formData) {
16     await new Promise(resolve => setTimeout(resolve, 2000)) // imagine que este es un request a una API por tu bien
17     console.log('Formulario enviado:', Object.fromEntries(formData))
18   }
19
20   return (
21     <form action={submit}>
22       <input name="email" type="email" required />
23       <SubmitButton />
24     </form>
25   )
26 }
```

Vamos por 
useFormStatus



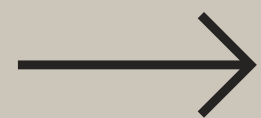
¿Cuándo usarlo?

- **Mostrar **spinners de carga** durante el envío**
- **Deshabilitar elementos del formulario (el mismo botón)**
- **Feedback en tiempo real a los usuarios**
- **Sólo en componentes hijos del formulario ⚠**



useDeferredValue

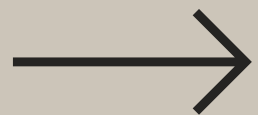
- **Evita que se "cuelgue" la página cuando escribes rápido en un buscador**
- **Mantiene el input responsivo aunque haya muchos resultados que mostrar**
- **Es como si le digo a React: "esto puede esperar, que el usuario pueda seguir escribiendo"**



Ejemplo de código

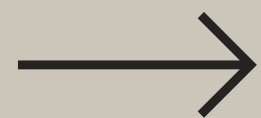
```
1  import { useDeferredValue, useState, useMemo } from 'react'
2
3  function Results({ consulta }) {
4    const deferredQuery = useDeferredValue(consulta)
5
6    const results = useMemo(() => {
7      return elements.filter(e =>
8        e.nombre.toLowerCase().includes(deferredQuery.toLowerCase())
9      ) // imagina que es una consulta larguísima a una API
10   }, [deferredQuery])
11
12   return (
13     <div>
14       {results.map(res => (
15         <div key={res.id}>{res.nombre}</div>
16       ))}
17     </div>
18   )
19 }
20
21 export default function SearchInput() {
22   const [consulta, setConsulta] = useState(''); // che una consulta
23
24   return (
25     <>
26       <input
27         value={consulta}
28         onChange={(e) => setConsulta(e.target.value)}
29         placeholder="Buscar..."
30       />
31       <Results consulta={consulta} />
32     </>
33   )
34 }
```

Vamos por 
useDeferredValue



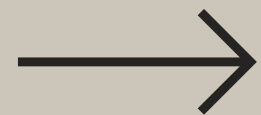
¿Cuándo usarlo?

- **Filtros de búsqueda** para listas costosas de renderizar
- **Mejorar el rendimiento** sin tener que usar el *debounce*
- **Representaciones gráficas** con cálculos complejos
- Interfaces que dependan de **valores que cambian frecuentemente**



useTransition

- **Evita que la página se "congele"** cuando cambias de pestaña o sección
- Te dice **cuándo algo está cargando** para mostrar un spinner o efecto visual
- Para **actualizaciones no urgentes**, que el usuario haga otras cosas mientras

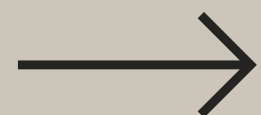


Ejemplo de código



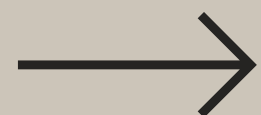
```
1  import { useState, useTransition } from 'react';
2
3  export default function TabComponent() {
4    const [activeTab, setActiveTab] = useState('home');
5    const [isPending, startTransition] = useTransition();
6
7    const handleTabChange = (tab) => {
8      startTransition(() => {
9        setActiveTab(tab);
10     });
11   };
12
13   return (
14     <div style={{opacity: isPending ? 0.7 : 1}}>
15       <div>
16         <button onClick={() => handleTabChange('home')}>Inicio</button>
17         <button onClick={() => handleTabChange('profile')}>Perfil</button>
18         <button onClick={() => handleTabChange('settings')}>Ajustes</button>
19       </div>
20
21       <div>
22         {activeTab === 'home' && <div>🏠 Contenido de Inicio</div>}
23         {activeTab === 'profile' && <div>👤 Contenido de Perfil</div>}
24         {activeTab === 'settings' && <div>⚙️ Contenido de Ajustes</div>}
25       </div>
26     </div>
27   );
28 }
```

Vamos por 
useTransition



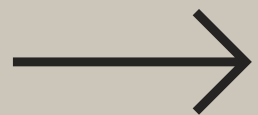
¿Cuándo usarlo?

- **Navegación** entre rutas
- **Renderizar un loader** mientras llega la respuesta del servidor
- Actualizaciones de **listas grandes**
- En general, para **cualquier actualización que requiera tiempo**



useOptimistic

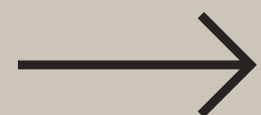
- Mi hook favorito para **mejorar la experiencia de usuario**
- **Muestra el cambio de estado inmediatamente** aunque no haya llegado al servidor
- Si algo sale mal, **se revierte solo** sin que tengas que hacer nada



Ejemplo de código

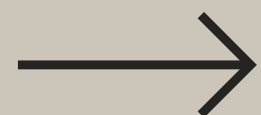
```
1 import { useState, useOptimistic } from 'react';
2
3 export default function LikeButton({ postId }) {
4   const [likes, setLikes] = useState(42);
5   const [optimisticLikes, addOptimisticLike] =
6     useOptimistic(likes, (state, increment) =>
7       state + increment
8     );
9
10  const handleLike = async () => {
11    addOptimisticLike(1);
12
13    try {
14      const response = await fetch(`/api/posts/${postId}/like`, {
15        method: 'POST'
16      }); // aqui iria tu llamada a la API de verdad
17      const data = await response.json();
18      setLikes(data.likes);
19    } catch (error) {
20      console.log('Error, se revierte el solito');
21    }
22  };
23
24  return (
25    <div>
26      <button onClick={handleLike}>
27        ❤️ {optimisticLikes}
28      </button>
29      <p>Haz clic y verás el cambio enseguida!</p>
30    </div>
31  );
32 }
```

Vamos por 
useOptimistic



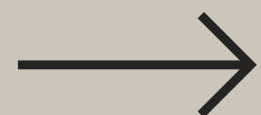
¿Cuándo usarlo?

- YouTube y Twitter (X) lo utilizan para sus **likes**
- También se puede aplicar para **comentarios, historial de cambios...**
- Cuando esperas una respuesta del servidor y **no ves necesario un loader pero quieres dar feedback**



useReducer

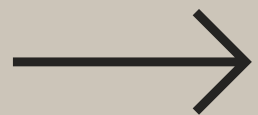
- **Evita el lío de varios useState cuando necesitas manejar datos relacionados**
- **Centraliza toda la lógica de un único estado**
- **Te ayuda a pensar en acciones como "incrementar", "resetear", "agregar item"...**



Ejemplo de código

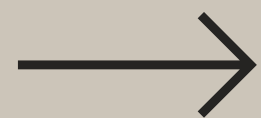
```
1  import { useReducer } from 'react';
2
3  function reducer(state, action) { // lo puedes separar de tu componente
4    switch (action.type) { // piensas en acciones, que te ayuda a entender mejor el código
5      case 'increment': return { count: state.count + 1 };
6      case 'decrement': return { count: state.count - 1 };
7      case 'reset': return { count: 0 };
8      default: return state;
9    }
10 }
11
12 export default function Counter() {
13   const [state, dispatch] = useReducer(reducer, { count: 0 }); // te queda muchísimo más limpio
14
15   return (
16     <div>
17       <p>Contador: {state.count}</p>
18       <button onClick={() => dispatch({ type: 'increment' })}>+</button>
19       <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
20       <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>
21     </div>
22   );
23 }
```

Vamos por 
useReducer



¿Cuándo usarlo?

- Muy útil en **formularios con lógica compleja** (multi-step, muchos campos...)
- Cuando ya tienes **10 useState para lo mismo**
- Quieres **mantener tu código más escalable y entendible**
- Si por algún casual, **te gusta Redux** (patrón similar)

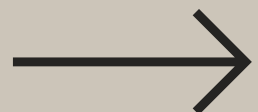


EXTRA

Te dejo un Custom Hook que utilizo en cada proyecto.
Si me preguntasen que me llevaría a una isla desierta,
siempre diría esto.

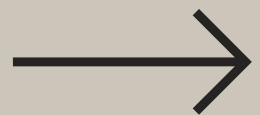


```
1  export function useFetch(endpoint) {  
2    const [data, setData] = useState(null);  
3    const [loading, setLoading] = useState(true);  
4    const [error, setError] = useState(null);  
5  
6    useEffect(() => {  
7      fetch(endpoint)  
8        .then(res => res.json())  
9        .then(setData)  
10       .catch(setError)  
11       .finally(() => setLoading(false));  
12    }, [endpoint]);  
13  
14    return { data, loading, error };  
15  }
```



Si te ha parecido útil, **guárdalo**
y **compártelo** con tu equipo o
tu yo del futuro

Creéme que te lo agradecerán 😊



SÍGUEME

para estar al loro
de consejitos en
programación



Adrián García Chavero

Construyo productos digitales
que dejan huella 🧠